Rent Price Prediction Using Linear Regression

Business Intelligence for Finance

Executive Summary

A linear regression model trained on the logarithm of rent predicts monthly rent for unseen listings with an R^2 of **0.70** (0.83 on the log scale) and an average error of about **32%**, which makes it a useful pricing guide but not an exact valuation tool.

This project analyses 4,746 rental listings from six Indian cities (Mumbai, Chennai, Bangalore, Hyderabad, Delhi and Kolkata). We explored the data, cleaned it, selected and transformed features, compared five regression models, and interpreted the results for decision-making.

Key findings

- Rent is strongly right-skewed (skewness 21.4). Taking the log reduced it to 0.9 and lifted test R^2 from 0.57 to 0.70 compared with a model on raw rent.
- City is the strongest driver of rent. Holding other features constant, a Mumbai property rents for about 181% more than the Bangalore baseline, while Kolkata is about 25% cheaper.
- Furnishing and size matter. Unfurnished homes rent for about 29% less than furnished ones, and larger, higher-BHK and higher-bathroom homes rent for more.
- Ridge, Lasso and ElasticNet performed almost identically to plain linear regression, so the model is not overfitting. Polynomial terms added complexity without a real benefit.

Recommendation. Use the linear model as a benchmark to judge whether a listing is fairly priced, always report a range rather than a single figure, and improve accuracy mainly by adding richer location data (neighbourhood, distance to transport, building age) rather than by tuning the model further.

1. Introduction

This project builds a linear regression model that predicts monthly rental prices from the physical and contractual characteristics of a property, and uses the results to support pricing decisions.

1.1 Background

Rent is one of the largest recurring expenses for households and the main source of income for landlords, so a fair and consistent way to price a property matters to tenants, landlords, agents and analysts alike. Listings differ in size, location, furnishing and many other features, which makes it hard to judge by eye whether an asking rent is reasonable. Business Intelligence (BI) methods allow these differences to be measured, so that rent can be explained and predicted from data rather than from intuition.

1.2 Objectives

1. Explore the dataset and understand the distribution of rent and its relationship with the available features.
2. Clean and prepare the data, including handling data-quality problems, outliers, categorical variables and feature scaling.
3. Select and transform the features that are most useful for predicting rent.
4. Build, tune and compare regression models, with a linear regression model as the main model.

5. Interpret the model coefficients and evaluate its performance on unseen data.
6. Turn the findings into practical insights and recommendations.

1.3 Scope and assumptions

- **Target variable.** The project follows the standard house-price prediction workflow (exploration, preprocessing, feature selection, modelling, evaluation), applied here to **rent**. The dataset contains rental listings, so the variable predicted is monthly rent.
- **Available features.** Some features often used in house-price studies, such as crime rate and distance to public transport, are not in this dataset. The model therefore uses only what is available: size, number of bedrooms (BHK), bathrooms, floor, building height, area type, furnishing status, tenant preference and city.
- **Currency.** The file does not state a currency. Because all six cities are in India, rent values are assumed to be in Indian rupees (INR); this assumption does not affect the analysis.
- **Time period.** Listings were posted over a short window of a few months in 2022, so the model cannot learn seasonal patterns or long-term price trends.

1.4 Tools and approach

The analysis was done in Python using Google Colab. The main libraries were pandas and NumPy for data handling, matplotlib and seaborn for charts, and scikit-learn for preprocessing, modelling and evaluation. The data was split 80% / 20% into training and test sets, and all preprocessing steps were fitted on the training data only to avoid data leakage.

2. Dataset Description

The dataset (House_Rent_Dataset.csv) contains **4,746 rental listings** and **13 columns**, posted between 13 April and 11 July 2022 across six cities, with no missing values and no duplicate listings.

2.1 Variables

Variable	Type	Description and summary
ID	Integer	Listing identifier (no predictive value)
Posted On	Date (text)	Date the listing was posted, 81 distinct dates
BHK	Integer	Number of bedrooms, hall and kitchen units; 1 to 6, mean 2.08
Rent	Integer	**Target variable**: monthly rent; 1,200 to 3,500,000, median 16,000, mean 34,993
Size	Integer	Property size, assumed to be square feet; 10 to 8,000, median 850, mean 967
Floor	Text	Unit floor and building height, e.g. "1 out of 3"; 480 distinct values
Area Type	Categorical	Super Area (2,446), Carpet Area (2,298), Built Area (2)
Area Locality	Text	Neighbourhood name; 2,235 distinct values

Variable	Type	Description and summary
City	Categorical	Mumbai (972), Chennai (891), Bangalore (886), Hyderabad (868), Delhi (605), Kolkata (524)
Furnishing Status	Categorical	Semi-Furnished (2,251), Unfurnished (1,815), Furnished (680)
Tenant Preferred	Categorical	Bachelors/Family (3,444), Bachelors (830), Family (472)
Bathroom	Integer	Number of bathrooms; 1 to 10, mean 1.97
Point of Contact	Categorical	Contact Owner (3,216), Contact Agent (1,529), Contact Builder (1)

2.2 Initial data-quality assessment

The first inspection of the data showed that it is complete but not yet ready for modelling:

- **Completeness.** All 13 columns have 4,746 non-missing values, and there are no duplicate rows once the ID column is ignored.
- **Skewed target.** The mean rent (about 35,000) is more than double the median (16,000), and the maximum (3,500,000) is far above the 75th percentile (33,000).
- **Suspicious values.** Size ranges from 10 to 8,000 sq ft. In total, 95 listings have a size below 100 or above 5,000 sq ft, which is unusual for a rental property.
- **Messy text columns.** Floor mixes words and numbers ("Ground out of 2", "Upper Basement", or a bare number), and Area Locality has more than 2,000 distinct values.
- **Rare categories.** Built Area appears in only 2 rows and Contact Builder in only 1 row, which is too few for a model to learn from.

Each of these issues is addressed in Section 4.

3. Data Exploration

Exploratory data analysis (EDA) shows that rent is extremely right-skewed, that city is the strongest separator of rent levels, and that size-related features (size, BHK, bathrooms) move together and all rise with rent.

3.1 Summary statistics

Variable	Mean	Std. dev.	Min	25%	Median	75%	Max
Rent	34,993	78,106	1,200	10,000	16,000	33,000	3,500,000
Size (sq ft)	967	634	10	550	850	1,200	8,000
BHK	2.08	0.83	1	2	2	3	6
Bathroom	1.97	0.88	1	1	2	2	10

Rent has a standard deviation more than twice its mean, and its maximum is over 100 times its median, which shows how strongly a small number of very expensive listings stretch the distribution.

3.2 Distribution of the target variable

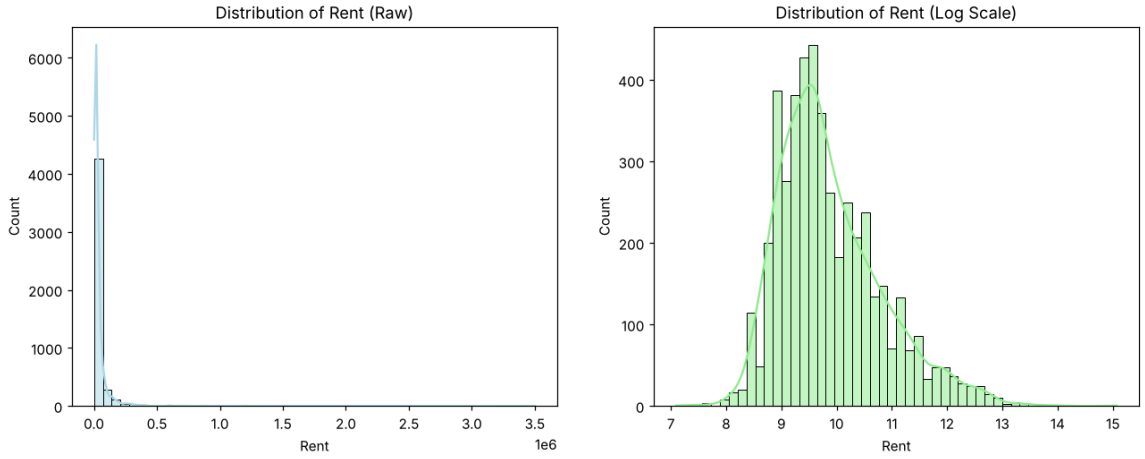


Figure 1. Distribution of rent: raw scale (left) and log scale (right).

On the raw scale almost all listings are crowded into the lowest bar, with a long tail stretching to 3.5 million. The skewness is **21.41**. After taking the natural logarithm of rent, the distribution is much closer to a symmetric bell shape, with a skewness of **0.91**. This matters because linear regression works best when errors are roughly symmetric and similar in size across the range of rent, which is why the model is trained on log(rent) (Section 5.3).

3.3 Numeric features and rent

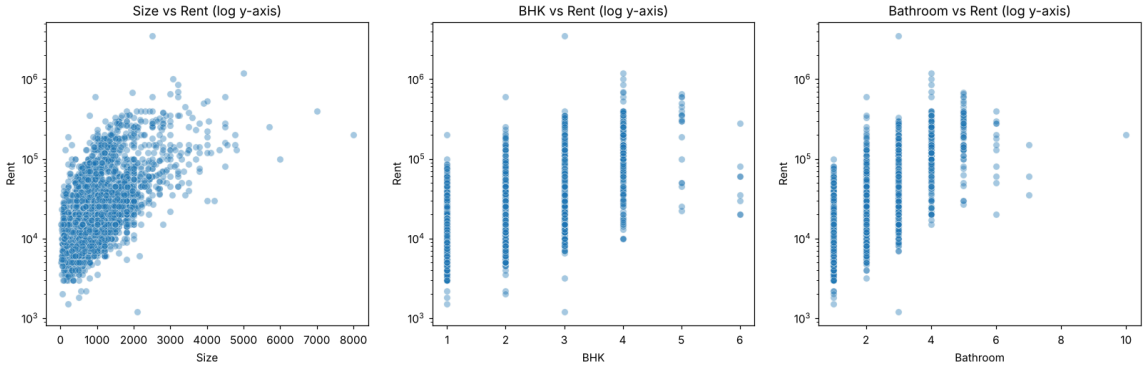


Figure 2. Size, BHK and Bathroom against rent (log y-axis).

Rent rises with size, with the number of bedrooms and with the number of bathrooms, although there is a wide spread of rent at every level. A few points with very small or very large sizes stand out from the main cloud and look like data-entry errors; they are handled in Section 4.4.

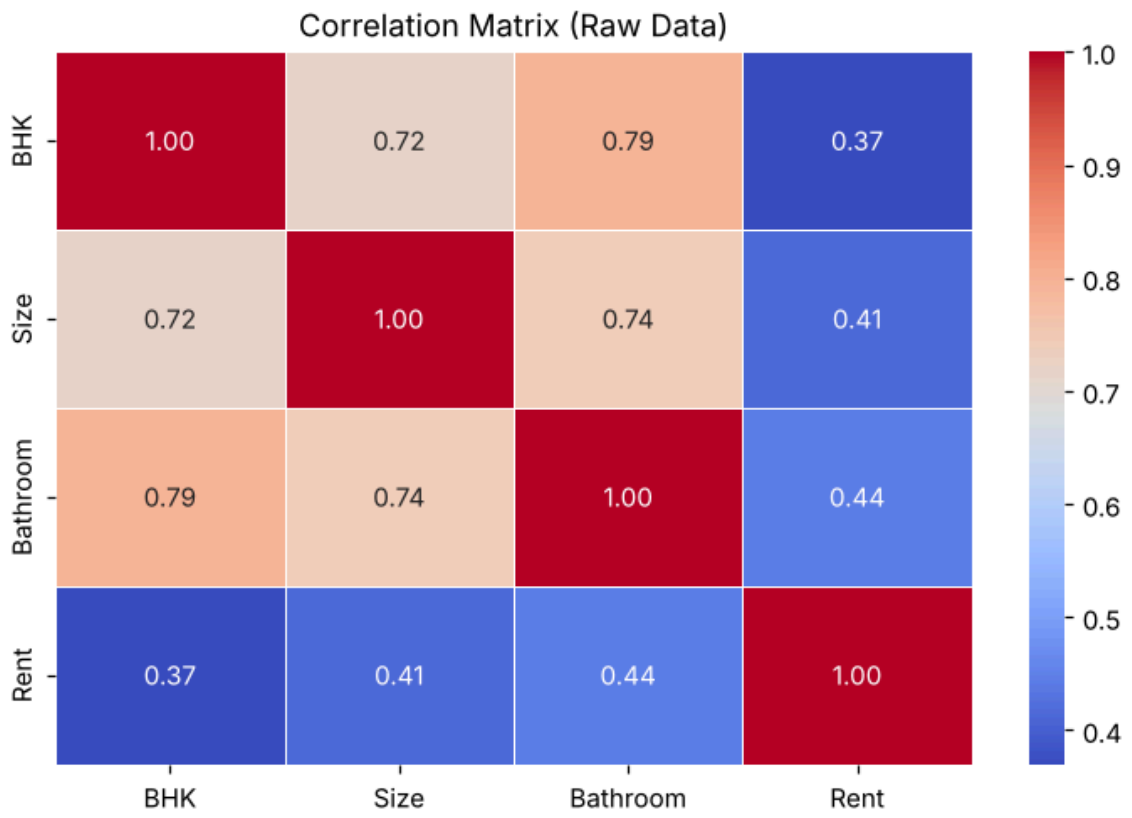


Figure 3. Pearson correlation matrix of the numeric variables (raw data).

Bathroom (0.44), Size (0.41) and BHK (0.37) are all positively correlated with raw rent. They are also strongly correlated with each other: BHK and Bathroom 0.79, Size and Bathroom 0.74, BHK and Size 0.72, which is natural because bigger homes have more rooms and bathrooms. Because of this overlap, the effect of any single size-related feature should be interpreted carefully, and multicollinearity is checked formally with VIF in Section 5.2. Raw correlations with rent are modest partly because rent is so skewed; they are higher once rent is logged (Section 5.1).

3.4 Categorical features and rent

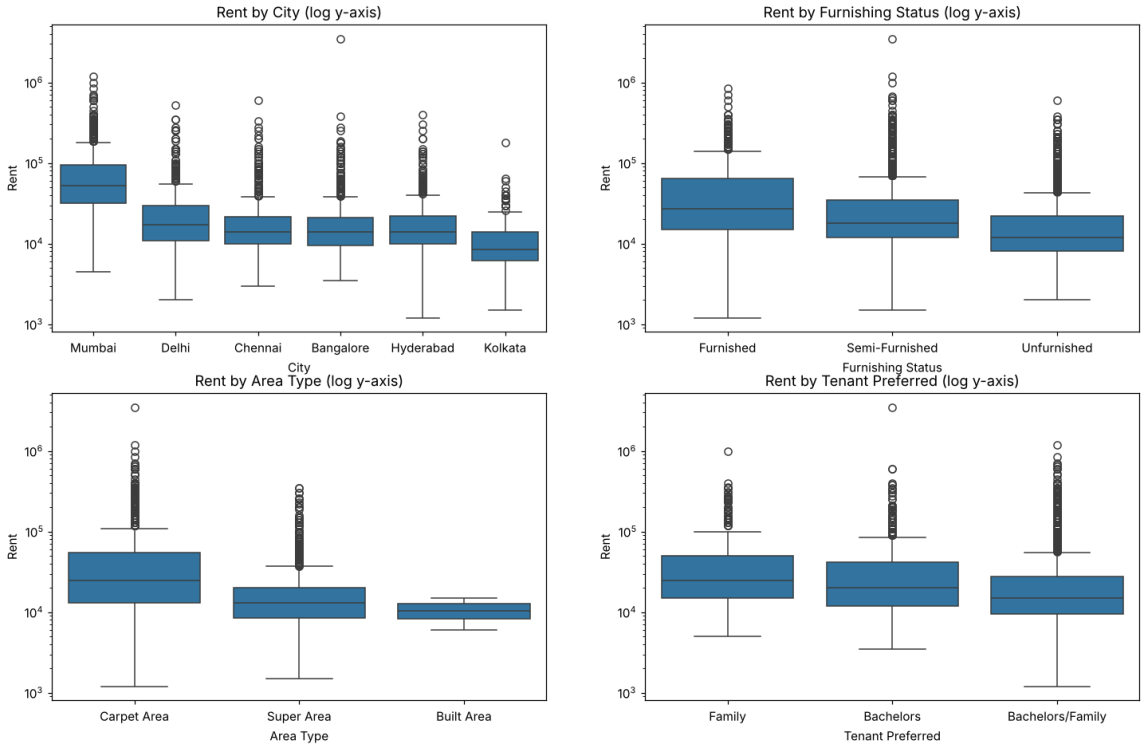


Figure 4. Rent by city, furnishing status, area type and tenant preferred (log y-axis).

City	Listings	Median rent	Mean rent
Mumbai	972	52,000	85,321
Delhi	605	17,000	29,462
Chennai	891	14,000	21,614
Bangalore	886	14,000	24,966
Hyderabad	868	14,000	20,555
Kolkata	524	8,500	11,645

- **City** has the largest effect. Mumbai's median rent (52,000) is more than three times that of Chennai, Bangalore or Hyderabad and about six times Kolkata's. Delhi is moderately higher than the three middle cities, and Kolkata is the cheapest.
- **Furnishing status** shows a clear ordering: furnished homes have the highest median, semi-furnished are in the middle and unfurnished are the lowest.
- **Area type.** Carpet Area listings have a visibly higher median than Super Area listings. This is partly a mix effect (which cities and property types use each measure) rather than a direct effect, so it is checked again in the model.
- **Tenant preferred.** Listings aimed at families have the highest median and "Bachelors/Family" the lowest, but the differences are smaller than for city or furnishing.

3.5 Key insights from exploration

1. Rent is heavily right-skewed, so a log transformation is needed.
2. City, furnishing status and the size-related features are the most promising predictors.
3. BHK, Size and Bathroom overlap strongly, so multicollinearity must be checked.
4. Some size and rent-per-square-foot values are implausible, so outlier handling is required.
5. Floor and Area Locality are messy text fields that need engineering or removal.

4. Data Preprocessing

Preprocessing turned the raw listings into a clean modelling dataset of 4,639 rows by checking and dropping weak columns, parsing the Floor text, removing 107 implausible records, and building a leakage-free scaling and encoding pipeline.

4.1 Checking Point of Contact before dropping it

Point of Contact (owner, agent or builder) looks like a strong predictor at first glance, because agent listings have a much higher median rent (45,000) than owner listings (12,500). The cause is largely the city, as agents handle most listings in the most expensive market:

City	Share of listings posted by an agent
Mumbai	80%
Delhi	40%
Bangalore	21%
Chennai	15%
Kolkata	15%
Hyderabad	13%

When Mumbai is excluded, the median rent of agent listings falls from 45,000 to 29,000 while owner listings stay at about 12,000, so a large part of the "agent premium" is really a city effect. Some difference remains, so the column is not purely a proxy for city. **Decision:** Point of Contact is dropped. City is already in the model, and keeping both would double-count location and make the city coefficients harder to interpret. Whether it adds anything beyond city is left as future work.

4.2 Parsing the Floor column

Floor holds text such as "3 out of 10" or "Ground out of 2". A custom function split it into two numeric features: the unit's own **Floor** (the number before "out of"; Ground = 0, Basement = -1) and **Total Floors** (the number after "out of"). Four listings have no "out of" part (for example a bare "1" or "Ground"), so their Total Floors is missing; these four values are filled with the median inside the training pipeline.

4.3 Dropping columns and grouping rare categories

Column	Decision	Reason
ID	Dropped	Identifier only

Column	Decision	Reason
Posted On	Dropped	Date metadata; the data covers only about three months, so there is little signal
Point of Contact	Dropped	Largely tied to city (Section 4.1)
Area Locality	Dropped	2,235 distinct values; one-hot encoding would create thousands of columns and label-encoding would imply a meaningless order
Area Type: Built Area	Merged into Carpet Area	Only 2 rows, too few to learn from
Floor	Parsed into Floor and Total Floors	Mixed text and numbers

4.4 Outlier handling

Outliers were identified with simple, fixed business rules rather than statistics taken from the whole dataset, so that no information from the future test set influences the cleaning. Two rules were used:

- **Implausible size:** size below 100 sq ft or above 5,000 sq ft (95 listings). A 20 sq ft flat renting for 5,000 is an example.
- **Implausible rent per square foot:** rent divided by size below 2 or above 300 (20 listings). Examples are a 2,100 sq ft home at a rent of 1,200 and a 2,500 sq ft home at 3,500,000, the single most extreme rent in the data.

In total **107 rows (2.3%)** were removed because some listings break both rules, leaving **4,639 rows**. Rent per square foot has a median of about 20 but a maximum of 1,500, which confirms that a few values are errors.



Figure 5. Rent after outlier removal. The remaining long right tail is genuine high-priced properties.

Expensive but plausible properties, such as large Mumbai apartments, were deliberately **kept**: they are real observations and removing them would make the model blind to the top of the market. The log transformation (Section 5.3) reduces their influence on the fitted model.

4.5 Train–test split

The cleaned data was split **80% / 20%** into a training set of **3,711 rows** and a test set of **928 rows**, with a fixed random seed (42) so results are reproducible. The split was made before any scaling, encoding or imputation, so the test set stays completely unseen. If preprocessing were fitted on the full dataset first, information from the test rows would leak into training and make the scores look better than they really are.

4.6 Scaling and encoding pipeline

All transformations were placed inside a scikit-learn pipeline that is fitted on the training data only and then applied unchanged to the test data.

Feature group	Variables	Treatment
Numeric	BHK, Size, Bathroom, Floor, Total Floors	Median imputation, then standardization (mean 0, standard deviation 1)
Categorical	Area Type, Furnishing Status, Tenant Preferred, City	Most-frequent imputation, then one-hot encoding with the first category dropped

- **Scaling.** Plain linear regression does not need scaling, but the Ridge, Lasso and ElasticNet penalties do, and scaling also makes coefficients comparable across features.
- **Categorical variables.** These have only 2 to 6 levels each, so one-hot encoding is suitable. Dropping the first category avoids the dummy-variable trap, and each dummy is read relative to that baseline: Carpet Area, Furnished, Bachelors and Bangalore. Unseen categories at prediction time are ignored rather than causing errors.

5. Feature Selection and Transformation

All nine remaining features (five numeric and four categorical) were kept, because each is related to rent and none shows a serious multicollinearity problem; the main transformation is taking the logarithm of rent.

5.1 Feature relevance

Feature relevance was checked on the **training data only**. For numeric features, the correlation with log(rent) was calculated:

Numeric feature	Correlation with log(rent)
Bathroom	0.693
BHK	0.600
Size	0.572
Total Floors	0.563
Floor	0.500

All five are moderately to strongly related to rent. These correlations are clearly higher than those with raw rent in Figure 3 (0.37 to 0.44), which is another sign that the log scale suits the data better. For categorical features, the median rent of each category was compared:

Feature	Median rent by category (training data)
City	Mumbai 52,000; Delhi 18,000; Bangalore 14,500; Hyderabad 14,500; Chennai 14,000; Kolkata 8,500
Furnishing Status	Furnished 27,500; Semi-Furnished 18,000; Unfurnished 12,000
Area Type	Carpet Area 25,000; Super Area 13,000
Tenant Preferred	Family 25,000; Bachelors 20,000; Bachelors/Family 15,000

City shows by far the largest differences between categories, followed by furnishing status. Area Type and Tenant Preferred also differ, although part of this reflects the cities and property sizes they happen to be associated with. All four categorical features were kept because they are cheap to include and their usefulness is confirmed by the model coefficients in Section 7.

5.2 Multicollinearity check (VIF)

Because BHK, Size and Bathroom are strongly correlated (Figure 3), the Variance Inflation Factor (VIF) was calculated for the numeric features on the training data. VIF measures how well each feature can be predicted from the others; values above about 5 to 10 signal a problem.

Feature	VIF
Total Floors	4.10
Floor	4.00
Bathroom	3.56
BHK	3.25
Size	2.65

All values are below 5, so multicollinearity is moderate rather than severe and no numeric feature had to be removed. Total Floors and Floor have the highest VIFs, which is expected because a unit's floor can never exceed the building height. VIF was used as a diagnostic rather than an automatic deletion rule, and it was not calculated for one-hot dummies, which are structurally related to each other by design.

5.3 Feature transformations

1. **Log transformation of rent (target).** Rent is extremely skewed (skewness 21.4; Section 3.2), and the same percentage mistake costs far more in absolute terms for an expensive property than for a cheap one. Modelling log(rent) makes the errors more even across the range and makes coefficients readable as percentage changes: a coefficient *b* means rent changes by about (e^b - 1) × 100%. Predictions are converted back with the exponential function so that errors are reported in rent units.
2. **Floor parsing.** The text column was turned into two numeric features (Section 4.2).
3. **Standardization and one-hot encoding.** Applied inside the pipeline (Section 4.6).

4. **Polynomial features (degree 2).** Squared and interaction terms were tested as a model extension (Section 6.4).

The benefit of the log transformation was tested directly. The same linear regression was trained once on raw rent and once on log(rent):

Linear regression trained on	R^2 on test set (rent scale)	MAE (rent units)	Negative rent predictions
Raw rent	0.5675	20,909	189 of 928 test listings
Log(rent)	0.7014	11,642	None possible

The log model explains far more variation, cuts the average error by about 44%, and cannot predict a negative rent, whereas the raw-rent model predicted a negative rent for about one listing in five.

5.4 Final feature set

- **Numeric (5):** BHK, Size, Bathroom, Floor, Total Floors.
- **Categorical (4):** City, Furnishing Status, Area Type, Tenant Preferred.
- **Target:** log(Rent).

6. Model Building

Five models were built and compared on the same held-out test set; plain linear regression performed as well as the regularized versions and clearly better than the polynomial model on the rent scale, so it was chosen as the final model.

6.1 Evaluation metrics

All models were trained on log(rent) and judged on the test set after converting predictions back to rent units. Five measures were used:

- **MAE** (mean absolute error): the average size of the error, in rent units.
- **RMSE** (root mean squared error): like MAE but punishes large errors more heavily.
- **MAPE** (mean absolute percentage error): the average error as a percentage of actual rent, the easiest measure to explain to non-technical readers.
- **R^2** on the rent scale: the share of variation in actual rent that the model explains.
- **R^2 on the log scale**: the same measure on the scale the model was trained on.

6.2 Linear regression (baseline and main model)

A linear regression was fitted inside the preprocessing pipeline on the 3,711 training rows. It reached an R^2 of 0.7014 on the test set (0.8297 on the log scale), an MAE of about 11,642 and a MAPE of 32.2%. Section 5.3 showed that the same model on raw rent performs much worse.

6.3 Model complexity, regularization and hyperparameter tuning

A model that is too flexible memorizes the training data and performs badly on new data. Regularization limits this by penalizing large coefficients: **Ridge** (L2 penalty) shrinks all coefficients, **Lasso** (L1 penalty) can shrink some exactly to zero, and **ElasticNet** mixes both. The strength of the penalty (alpha) and, for ElasticNet, the mix (l1_ratio) are hyperparameters. They were tuned with **GridSearchCV and 5-fold**

cross-validation on the training set only, scored by mean squared error on the log scale. The preprocessing sits inside the pipeline, so it is re-fitted within each fold and no test information is used.

Model	Values searched	Best setting
Ridge	alpha: 0.01, 0.1, 1, 10, 100	alpha = 1
Lasso	alpha: 0.0001, 0.001, 0.01, 0.1, 1	alpha = 0.0001
ElasticNet	alpha: 0.0001 to 1 (5 values); l1_ratio: 0.2, 0.5, 0.8	alpha = 0.0001, l1_ratio = 0.8

The best penalties are small, meaning the search preferred almost no shrinkage, and all three regularized models performed almost identically to plain linear regression (Section 6.5). This is expected: the model has only 15 inputs after encoding and 3,711 training rows, so there is little overfitting for regularization to fix.

6.4 Polynomial regression (degree 2)

To test whether a more flexible model helps, squared terms and pairwise interactions (for example Size × a city) were added to the preprocessed features and fitted with a mild Ridge penalty (alpha = 10) to keep it stable. On the log scale the polynomial model is almost the same as the linear one, and its training R^2 (0.8325) is close to its test R^2 (0.8307), so it does not overfit badly. However, on the rent scale it is clearly worse (R^2 0.4358, RMSE 43,394 against 31,568 for linear). A few extreme over-predictions on the log scale become very large errors once converted back to rent units. For example, the polynomial model predicts about 1,029,000 for a listing that actually rents at 230,000 and about 1,209,000 for one at 500,000, and five listings account for about 78% of its squared error (against about 45% for the linear model). The polynomial model is therefore not plotted in Figure 7, which shows the final linear model only.

6.5 Cross-validation stability

Five-fold cross-validation on the training set (R^2 on the log scale) gave:

Model	CV R^2 (mean ± std. dev.)
Linear regression	0.8093 ± 0.0101
Polynomial (degree 2, Ridge)	0.8200 ± 0.0134

The polynomial model scores about 0.01 higher, but the ranges overlap, so the difference is modest rather than significant. Together with its worse test results on the rent scale, this means the simpler linear model is the better choice.

6.6 Model comparison on the test set

Model	MAE	RMSE	MAPE	R^2 (rent scale)	R^2 (log scale)
Linear regression	11,642	31,568	32.21%	0.7014	0.8297
Ridge	11,642	31,552	32.20%	0.7017	0.8298
Lasso	11,638	31,545	32.20%	0.7019	0.8297

Model	MAE	RMSE	MAPE	R^2 (rent scale)	R^2 (log scale)
ElasticNet	11,639	31,548	32.20%	0.7018	0.8297
Polynomial (degree 2, Ridge)	11,914	43,394	32.42%	0.4358	0.8307

The four linear models are practically tied (the differences are far smaller than the cross-validation variation), so the simplest, **linear regression, is selected as the final model**: it is the easiest to explain, its coefficients can be interpreted directly, and it generalizes well.

7. Interpretation and Evaluation

The final linear model shows that location dominates rent, followed by furnishing and size, and it explains about 70% of the variation in rent on unseen listings with an average error of about 32%.

7.1 Interpreting the coefficients

Because the target is log(rent), each coefficient *b* converts to a percentage effect of (e^b - 1) × 100%. Numeric features were standardized, so their coefficient is the effect of a **one-standard-deviation increase**. Dummy variables are compared with the baseline category (Bangalore, Furnished, Carpet Area, Bachelors). All effects hold the other features constant.

Feature	Coefficient	Effect on rent
City: Mumbai (vs Bangalore)	1.032	+181%
Size (+1 std. dev.)	0.307	+36%
Furnishing: Unfurnished (vs Furnished)	-0.341	-29%
City: Kolkata	-0.286	-25%
City: Delhi	0.232	+26%
Furnishing: Semi-Furnished	-0.225	-20%
BHK (+1 std. dev.)	0.170	+19%
City: Hyderabad	-0.163	-15%
Area Type: Super Area (vs Carpet Area)	-0.160	-15%
Bathroom (+1 std. dev.)	0.120	+13%
Tenant Preferred: Family (vs Bachelors)	-0.119	-11%
Total Floors (+1 std. dev.)	0.081	+8%
Tenant Preferred: Bachelors/Family	-0.057	-6%
City: Chennai	-0.036	-4%
Floor (+1 std. dev.)	0.007	+1%

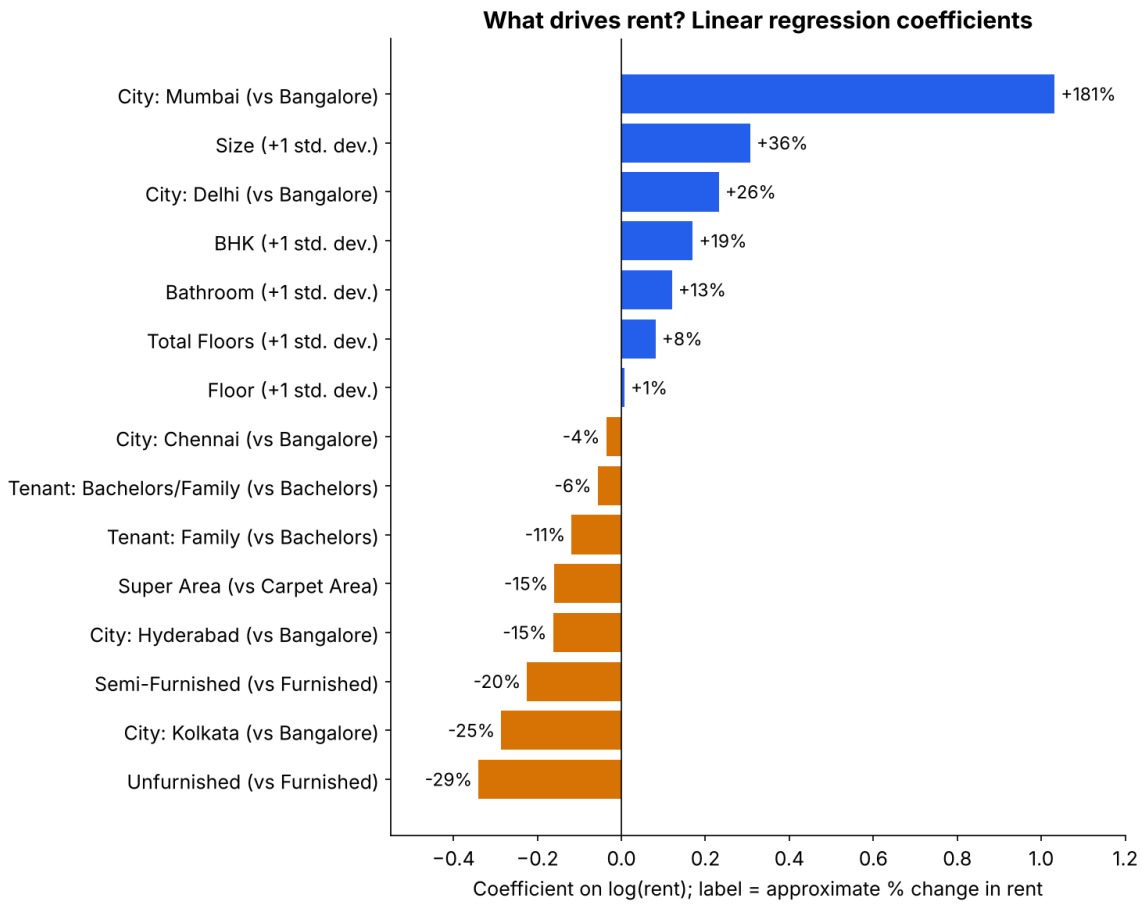


Figure 6. Linear regression coefficients (log-rent scale). Blue bars raise rent and orange bars lower it; each label is the approximate % change in rent, and the baseline category is shown in brackets.

The main messages are:

- **Location is the strongest driver.** A Mumbai property rents for about 181% more than an otherwise similar one in Bangalore, Delhi about 26% more, and Kolkata about 25% less. Chennai is almost the same as Bangalore.
- **Furnishing matters.** Unfurnished homes rent for about 29% less than furnished homes and semi-furnished homes about 20% less.
- **Bigger homes cost more.** One standard deviation more size, BHK or bathrooms is associated with roughly 36%, 19% and 13% higher rent. These three overlap strongly, so their effects should be read together rather than separately.
- **Super Area listings** rent for about 15% less than Carpet Area listings, a family preference is associated with about 11% less rent than a bachelor preference, and building height has a small positive effect.
- **The unit's own floor has almost no effect** (about +1%) once building height and the other features are included.

These coefficients describe associations, not causes. For example, furnishing a flat will not necessarily raise its rent by the full amount shown, because furnished homes may also differ in ways the data does not record.

7.2 Performance on the test set

The final linear regression, evaluated on 928 test listings that were never used for preprocessing, tuning or feature decisions, achieved:

Metric	Value	Meaning
R^2 (rent scale)	0.7014	About 70% of the variation in actual rent is explained
R^2 (log scale)	0.8297	About 83% explained on the scale the model was trained on
MAE	11,642	Average error in rent units
RMSE	31,568	Error measure that punishes big misses; larger than MAE because a few expensive listings are badly missed
MAPE	32.21%	Average error as a share of actual rent

R^2 is lower on the rent scale than on the log scale because rent-scale errors are dominated by the few very expensive listings, where a modest percentage error becomes a large absolute one.

7.3 Predicted versus actual rent and residuals

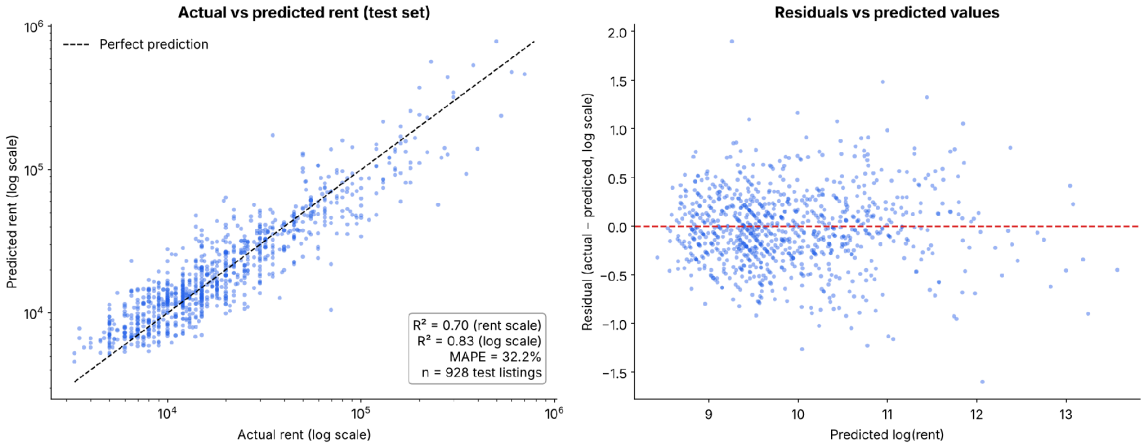


Figure 7. Left: actual against predicted rent for the test set (log axes); the dashed line is perfect prediction. The box summarizes R^2, MAPE and the number of test listings. Right: residuals of the linear model against predicted log(rent).

In the left chart the predictions follow the dashed line across a range of rent from about 3,000 to more than 500,000, so the model captures the overall pattern well, but individual predictions scatter around the line. In the right chart the residuals are centred on zero with no strong curve, and their spread is fairly even across predicted values, which supports using a linear model on the log scale. A small number of listings have large residuals (up to about 1.9 above and 1.6 below on the log scale), meaning some properties are priced very differently from what their recorded features suggest.

7.4 Where the model is more and less accurate

City	Median percentage error
Bangalore	20.9%
Mumbai	21.1%
Hyderabad	23.9%
Kolkata	24.7%
Chennai	25.4%
Delhi	28.8%

The typical (median) error is roughly 21% to 29% in every city, so the model is not dramatically better in one city than another. It is a little less accurate in Delhi; the data cannot show why, but missing neighbourhood detail is one possible reason.

7.5 Implications for predicting rent

- The model is reliable enough to **benchmark** a listing: a rent far above the predicted value may be overpriced, and one far below may be a bargain.
- An average error of about 32% is too large for exact valuation, so predictions should be presented as a **range**, not a single number.
- The remaining error shows that important drivers are missing from the data, such as the exact neighbourhood, building age, amenities and distance to transport.

8. Insights and Recommendations

The analysis supports using the model as a pricing benchmark that is driven mainly by city, size and furnishing, while treating its predictions as ranges and investing in better location data.

8.1 Key insights

1. **Rent depends first on where the property is.** Mumbai's median rent (52,000) is about six times Kolkata's (8,500), and after accounting for size and furnishing a Mumbai home is still about 181% more expensive than a comparable one in Bangalore.
2. **Furnishing carries a clear premium.** Unfurnished homes rent for about 29% less and semi-furnished homes about 20% less than furnished ones.
3. **Size-related features raise rent together.** Size, BHK and bathroom count are strongly linked, so rent rises as a bundle with property size.
4. **Pricing should be thought of in percentages.** Rent is highly skewed, so a model on the log scale describes the market far better than one on absolute rent.
5. **A simple model is enough with these data.** Adding regularization or polynomial terms gave no real gain, which means the limit on accuracy comes from missing information, not from the modelling method.

8.2 Recommendations

Audience	Recommendation
Landlords and agents	Price mainly by city, size and furnishing. Compare the extra rent a furnished listing can earn (about 20% to 29% more in this data) with the cost of furnishing, keeping in mind that the data shows association, not proof of cause.
Tenants and analysts	Use the model's prediction as a fair-price benchmark when negotiating or screening listings. A listing far above its predicted rent deserves a closer look.
Model owners	Report a prediction range instead of a single figure, because the average error is about 32%. Retrain regularly, since rents change over time and this dataset covers only about three months.
Data collection	Add neighbourhood-level location detail (coordinates or locality quality), building age, amenities and distance to public transport, which are the most likely sources of the remaining error.

8.3 How the model supports financial decision-making

In a Business Intelligence setting, the model turns many listings into a consistent pricing rule. A landlord or property manager can use it to set an opening rent, an investor can use it to compare the typical rent levels of cities and furnishing strategies, and a tenant can use it to judge whether an offer is fair. Its value lies in giving a transparent, explainable starting point, not in replacing local market knowledge.

9. Additional Challenges, Limitations and Future Work

Seven data challenges were found and handled, and the main remaining limits are the missing location and building information and the short time window of the data.

9.1 Challenges found and how they were handled

Challenge	How it was handled
Highly skewed rent (skewness 21.4)	Modelled log(rent); this lifted test R^2 from 0.5675 to 0.7014 and removed negative predictions
Outliers and data-entry errors	Removed 107 rows (2.3%) with implausible size or rent per square foot; kept expensive but plausible listings
Messy Floor column	Parsed into unit floor and total floors; 4 missing values filled with the median in the pipeline
High-cardinality locality (2,235 values)	Excluded from the model because one-hot encoding is impractical; listed as future work

Challenge	How it was handled
Correlated size features (BHK, Size, Bathroom)	Checked with VIF (all below 5) and interpreted together
Rare categories (Built Area 2 rows, Contact Builder 1 row)	Built Area merged into Carpet Area; Point of Contact dropped
Risk of data leakage	Split first, then fitted all preprocessing inside pipelines on training data only

9.2 Limitations

- **Association, not causation.** Coefficients show how rent differs between otherwise similar listings; they do not prove that changing a feature changes rent.
- **Short time window.** Listings were posted between 13 April and 11 July 2022, so the model cannot capture seasonality or longer-term price trends, and it should not be used for much later years without retraining.
- **Asking rents.** The data are listing prices, which may differ from the rent finally agreed.
- **Missing drivers.** Neighbourhood quality, building age, amenities and transport access are not in the data, which explains much of the remaining error of about 32%.
- **Assumed units.** The currency (INR) and the size unit (square feet) are assumed because the file does not state them.
- **Outlier rules are judgement-based.** The thresholds were chosen from the data's own plausibility checks; different thresholds would remove slightly different rows and change results a little.

9.3 Strategies to improve the model

1. **Use locality information.** Encode Area Locality with target encoding or geographic coordinates, with the encoding fitted inside cross-validation so that no test information leaks in.
2. **Try non-linear models.** Random Forest or Gradient Boosting could be compared with the linear model under the same leakage-free procedure.
3. **Model cities separately.** The effect of size on rent may differ between cities, so a separate model per city could be tested.
4. **Add missing drivers.** Building age, amenities and distance to public transport are likely to reduce the error more than any further tuning.
5. **Report uncertainty.** Provide prediction intervals or ranges alongside each predicted rent.
6. **Revisit Point of Contact.** Test whether it adds explanatory power beyond city once the other features are included.

10. Conclusion

This project showed that a simple linear regression on log(rent) can explain about 70% of the variation in rent for unseen listings, with an average error of about 32%, using only a handful of property features.

The work followed the full analytics workflow. Exploration revealed a heavily skewed target and a strong city effect. Preprocessing cleaned the data (107 implausible rows removed, Floor parsed, weak columns dropped) and built a leakage-free pipeline. Feature checks confirmed that all nine remaining features were useful and not seriously collinear. Five models were compared, and plain linear regression was chosen

because Ridge, Lasso and ElasticNet gave no real improvement and the polynomial model was clearly worse on the rent scale.

The most important drivers of rent are **city** (Mumbai about 181% above Bangalore, Kolkata about 25% below), **furnishing** (unfurnished about 29% below furnished) and **property size** (size, BHK and bathrooms). The model is best used as a pricing guide with a range of values rather than an exact valuation. The next gains will come mainly from richer data, especially neighbourhood-level location, building age and distance to public transport, rather than from further model tuning.

Appendix

A. Reproducibility

Item	Detail
Environment	Google Colab, Python with pandas, NumPy, matplotlib, seaborn and scikit-learn
Data file	House_Rent_Dataset.csv (4,746 rows, 13 columns)
Cleaned dataset	4,639 rows after removing 107 outliers
Split	80% training (3,711 rows), 20% test (928 rows), random_state = 42
Target	log(Rent); predictions converted back with the exponential function
Numeric features	BHK, Size, Bathroom, Floor, Total Floors
Categorical features	City, Furnishing Status, Area Type, Tenant Preferred
Cross-validation	5-fold, on the training set only

Results can differ very slightly between software versions, but the conclusions do not change.

B. Glossary

Term	Meaning
BHK	Number of bedrooms, hall and kitchen
Log transformation	Replacing a value by its natural logarithm, which compresses very large values
One-hot encoding	Turning a category into separate 0/1 columns
Standardization	Rescaling a feature to mean 0 and standard deviation 1
VIF	Variance Inflation Factor, a measure of how much a feature overlaps with the others
Regularization	Penalizing large coefficients to reduce overfitting (Ridge, Lasso, ElasticNet)

Term	Meaning
Cross-validation	Repeatedly training on part of the training data and testing on the rest to check stability
MAE, RMSE, MAPE	Average absolute error, root mean squared error and average percentage error
R^2	Share of the variation in the target that the model explains